

C++ and Robotics Interview Tutorial

A complete, self-contained study path for the six phases in `cpp-interview-tracker.html`. Every problem and concept in the tracker has a section here and a compilable, self-testing C++ file under `code/`. Read a chapter, type the solutions yourself, then mark the tracker.

The tutorial assumes you know Python and nothing about C++. Every idea starts with a picture or an everyday story, then a small example worked by hand, then the Python you would have written, then the C++ with every new symbol explained. Every problem starts with the full problem in plain words and a drawn example before any code appears.

Chapters

#	Chapter	Tracker phase	What you get
01	C++ basics	C++ basics	What C++ is, how to compile and run, types, functions, loops, vector, string, maps, pairs, 10 warm-up problems
02	Memory, objects and the STL	C++ interviewers test	Pointers and references drawn as boxes, const, classes and RAII, smart pointers, stack vs heap, the <code>size_t</code> trap, sort/heap/queue/map, 8 array/hash/interval problems
03	Grids and BFS/DFS	Grids and BFS/DFS	Grids, queues, stacks and recursion from zero, BFS and DFS with hand traces, backtracking, grid DP, 10 grid problems, A* built from scratch with tests
04	Graphs and Dijkstra	Graphs and Dijkstra	Graphs as cities and roads, BFS/DFS on graphs, topological sort, union-find, Dijkstra traced by hand, 8 graph problems
05	Heaps, trees, design	Heaps, trees, design	Heaps drawn as trees and arrays, top-k, sliding windows, binary trees, LRU cache as a bookshelf, 8 problems
06	Robotics-specific	Robotics-specific	Kalman filter as blending two guesses, PID as cruise control, rotations and frames with numbers, RANSAC, two premium robot problems, portfolio and CV guidance
07	Interview day	(all)	Read this last: how a live C++ round runs, how to talk while coding, a question bank with short answers, the last week

System design

The [system-design/](#) folder is a separate five-level path, from "what is a system" to full robotics design questions worked end to end. Start it after chapter 03, one level per sitting.

PDF versions

Every chapter and every system design level is also rendered as a PDF under `pdf/`, with the figures at fixed width so nothing wraps. `pdf/tutorial-book.pdf` and `pdf/system-design-book.pdf` are the combined books. To rebuild after editing any markdown file:

```
python3 pdf/build_pdfs.py
```

How to use the code

Every .cpp file under code/<chapter>/ is standalone. It contains the solution and a main() with assert-based tests and prints OK <name> when it passes.

```
cd "code"
make           # build and run everything, including the A* GoogleTest suite
make 03        # only chapter 03
make astar     # only the A* project
```

To compile a single file by hand (this is the command you should be comfortable typing):

```
g++ -std=c++17 -Wall -Wextra -O2 01-cpp-basics/01_two_sum.cpp -o two_sum && ./two_sum
```

Eigen files (chapter 06) need the include path:

```
g++ -std=c++17 -Wall -Wextra -O2 -isystem /usr/include/eigen3 06-robotics/03_transforms_eigen.cpp -o t && ./t
```

For debugging, swap -O2 for -g -fsanitize=address,undefined. The sanitizers catch out-of-bounds reads, use-after-free and signed overflow at run time. Interviewers like hearing that you use them.

The study method that actually works

1. **Read the concept part** of a chapter (Part A) slowly, and do every hand trace on paper yourself before reading on. Say each "In your own words" summary from memory.
2. **For each problem, close the tutorial and try it in C++ first**, timed, 25 minutes. Compile with -Wall -Wextra. Fix every warning.
3. **Only then read the solution.** Compare. If you needed it, mark the tracker "needed help" and redo the problem three days later from a blank file.
4. **Mark a concept done only when you can explain it to someone else** without notes. The "Check yourself" questions at the end of each chapter are the test.
5. **Type, do not copy-paste.** Muscle memory for std::vector<std::vector<int>>, auto& [k, v], std::priority_queue<..., std::greater<>> is exactly what a live round tests.

Six-week plan (about 2 hours a day)

Week	Focus	Target on the tracker
1	Chapter 01, all of it	C++ basics 18/18
2	Chapter 02 concepts + problems	C++ interviewers test 19/19
3	Chapter 03 problems 1-10	Grids 10/11
4	A* project (3 days) + Chapter 04 traversal problems	Grids 11/11, Graphs 4/8
5	Chapter 04 shortest paths + Chapter 05	Graphs 8/8, Heaps 8/8
6	Chapter 06 builds + portfolio + Chapter 07 drills	Robotics 9/9, overall "Ready"

If you have less time: weeks 1, 2, 3 and the Dijkstra template from chapter 04 are the minimum that gets you through a typical UK robotics coding round. Chapter 06 is what gets you through the robotics-specific round and what makes your CV stand out.

Readiness standard (same as the tracker)

- All C++ basics and interviewer-tested concepts done.
- At least 80% of problems done, at least 60% of all problems solved unaided in 25 minutes.
- A* and Dijkstra written from scratch without looking anything up.

Directory layout

```
tutorial/          the chapters (markdown)
code/
  Makefile         build + run everything
  01-cpp-basics/   one .cpp per problem, plus 00_*.demo.cpp for concepts
  02-cpp-interviewers-test/
  03-grids-bfs-dfs/
    astar/         standalone CMake project with GoogleTest (portfolio piece)
  04-graphs-dijkstra/
  05-heaps-trees-design/
  06-robotics/     Kalman, PID, Eigen transforms, RANSAC, premium problems
```